PasswordStore Security Audit Report

wildanfh

2026-04-21

PasswordStore Security Audit Report

Prepared by: wildanfh
Lead Auditor: wildanfh

Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time-boxed security review of the PasswordStore smart contract, and does not guarantee the absence of bugs or vulnerabilities. I make no warranties regarding the security of the code and am not liable for any damages related to the use of the code. The findings represent my best efforts during the allotted time.

Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	High	High/Medium	Medium
Likelihood: Medium	High/Medium	Medium	Medium/Low
Likelihood: Low	Medium	Medium/Low	Low

Protocol Summary

PasswordStore is a smart contract designed to allow the owner to store and retrieve a private password. Only the owner should be able to set and get the password. Others should not be able to access it.

Audit Details

Commit Hash: (see repository)

Scope:

```
#-- src/
    #-- PasswordStore.sol
```

Roles: - Owner: The user who deploys the contract. Can set and retrieve the password.

Executive Summary

During the audit of PasswordStore, 3 findings were identified: 2 High severity and 1 Informational. The most critical issues are the absence of access control on `setPassword` and the fundamental flaw of storing sensitive data on-chain, where all storage is publicly readable.

Summary of Findings

ID	Title	Severity
H-1	`PasswordStore::setPassword` has no access controls	High
H-2	Storing the password on-chain makes it visible to anyone	High
I-1	`PasswordStore::getPassword` natspec indicates a non-existent parameter	Informational

Severity	Count
High	2
Medium	0
Low	0
Informational	1

Findings

[H-1] `PasswordStore::setPassword` has no access controls, meaning a non-owner could change the password

Description: The `PasswordStore::setPassword` function is set to be an `external` function, however the purpose of the smart contract and function's natspec indicate that `This function allows only the owner to set a new password.`

```
function setPassword(string memory newPassword) external {
    // @Audit - There are no Access Controls.
    s_password = newPassword;
    emit SetNetPassword();
}
```

Impact: Anyone can set/change the stored password, severely breaking the contract's intended functionality.

Proof of Concept: Add the following to the `PasswordStore.t.sol` test file:

Code

```
function test_anyone_can_set_password(address randomAddress) public {
    vm.assume(randomAddress != owner);
    vm.startPrank(randomAddress);
    string memory expectedPassword = "myNewPassword";
    passwordStore.setPassword(expectedPassword);

    vm.startPrank(owner);
    string memory actualPassword = passwordStore.getPassword();
    assertEq(actualPassword, expectedPassword);
}
```

Run with:

```
forge test --mt test_anyone_can_set_password
```

Through 256 fuzz runs, any random address was able to call `setPassword` and override the owner's password.

Recommended Mitigation: Add an access control check to `PasswordStore::setPassword`:

```
function setPassword(string memory newPassword) external {
    if (msg.sender != s_owner) {
        revert PasswordStore__NotOwner();
    }
    s_password = newPassword;
    emit SetNetPassword();
}
```

[H-2] Storing the password on-chain makes it visible to anyone and no longer private

Description: All data stored on chain is public and visible to anyone. The `PasswordStore::s_password` variable is intended to be hidden and only accessible by the owner through the `PasswordStore::getPassword` function. I show one such method of reading any data off chain below.

Impact: Anyone is able to read the private password, severely breaking the functionality of the protocol.

Proof of Concept: The below steps show how anyone can read the password directly from the blockchain using Foundry's `cast` tool, without being the owner.

1. Create a locally running chain:

```
make anvil
```

2. Deploy the contract to the chain:

```
make deploy
```

3. Run the storage tool (slot 1 is the storage slot of `PasswordStore::s_password`):

```
cast storage <ADDRESS_HERE> 1 --rpc-url http://127.0.0.1:8545
```

You'll get an output that looks like:

```
0x6d7950617373776f72640000000000000000000000000000000000000000000014
```

4. Parse the hex to a string:

```
cast parse-bytes32-string 0
    x6d7950617373776f72640000000000000000000000000000000000000000000014
```

Output:

```
myPassword
```

Recommended Mitigation: Due to this, the overall architecture of the contract should be rethought. One could encrypt the password off-chain, and then store the encrypted password on-chain. This would require the user to remember another password off-chain to decrypt the stored password. However, you would also likely want to remove the view function as you wouldn't want the user to accidentally send a transaction with this decryption key.

[I-1] The `PasswordStore::getPassword` natspec indicates a parameter that doesn't exist, causing the natspec to be incorrect

Description:

```
/*
 * @notice This allows only the owner to retrieve the password.
@> * @param newPassword The new password to set.
 */
function getPassword() external view returns (string memory) {}
```

The `PasswordStore::getPassword` function signature is `getPassword()` while the natspec says it should be `getPassword(string)`. There is no parameter in the function.

Impact: The natspec is incorrect.

Recommended Mitigation: Remove the incorrect natspec line.

```
    /*
     * @notice This allows only the owner to retrieve the password.
-    * @param newPassword The new password to set.
     */
```